

Capítulo 7. Del hardware al usuario: comprendiendo las capas del Sistema Operativo

Capítulo 7. Capas del Sistema Operativo

Relación con la clase anterior

En el capítulo anterior aprendimos que el **Scheduler** decide qué proceso utilizará el procesador y que el **Kernel** coordina esa decisión.

Ahora responderemos una nueva pregunta:

¿Qué ocurre dentro de una computadora desde que hacemos doble clic sobre una aplicación hasta que aparece en pantalla?

Esta pregunta nos permitirá comprender cómo trabajan juntas las diferentes capas de un Sistema Operativo.

Objetivos de aprendizaje

Al finalizar este capítulo serás capaz de:

- Comprender la organización por capas de un Sistema Operativo.
 - Explicar la función de cada capa.
 - Identificar el papel del Kernel dentro de esa arquitectura.
 - Relacionar las capas con acciones que realizamos diariamente en una computadora.
-

Motivación

Todos los días abrimos aplicaciones como:

- Google Chrome
- Microsoft Word
- Visual Studio Code
- Spotify

La aplicación aparece en pocos segundos.

Pero...

- ¿Quién encuentra el programa en el disco?
- ¿Quién reserva memoria para ejecutarlo?
- ¿Quién se comunica con el teclado y el mouse?
- ¿Quién envía la imagen al monitor?
- ¿Quién controla realmente todo el proceso?

La respuesta está en las **capas del Sistema Operativo**.

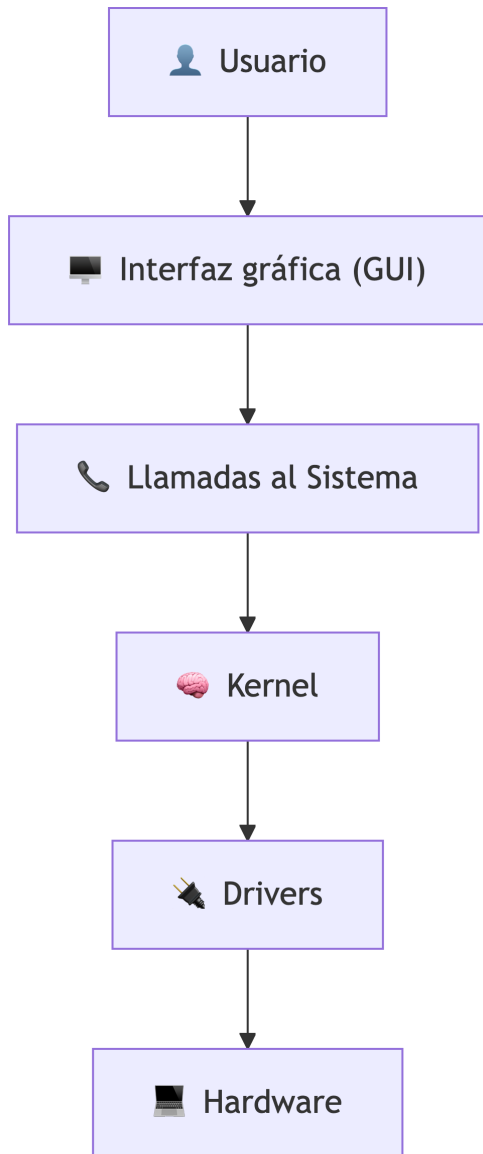
Pregunta detonadora

Cuando haces doble clic sobre una aplicación, ¿qué sucede dentro de la computadora antes de que aparezca la ventana?

Al finalizar este capítulo podrás responder esa pregunta paso a paso.

Un viaje por el Sistema Operativo

Durante esta clase seguiremos el recorrido que realiza una solicitud desde el usuario hasta el hardware.



Observa que el usuario nunca se comunica directamente con el hardware.
Siempre existe una cadena de componentes que trabajan coordinadamente.

¿Por qué organizar el Sistema Operativo por capas?

Dividir el Sistema Operativo en capas ofrece varias ventajas.

- Cada capa tiene una responsabilidad específica.
- Facilita el mantenimiento del sistema.
- Reduce errores.
- Permite actualizar componentes sin modificar todo el Sistema Operativo.
- Hace posible reutilizar código.

Piensa en un edificio.

Cada piso tiene una función distinta, pero todos trabajan juntos para que el edificio funcione correctamente.

Con un Sistema Operativo ocurre exactamente lo mismo.

¿Qué aprenderemos en este capítulo?

En las siguientes secciones estudiaremos:

1. La arquitectura por capas.
2. La función de cada capa.
3. El Kernel como corazón del Sistema Operativo.
4. Los tipos de Kernel.
5. Un caso práctico siguiendo el recorrido de una aplicación desde el usuario hasta el hardware.

Preparándonos para la siguiente sección

Ya conocemos el recorrido general.

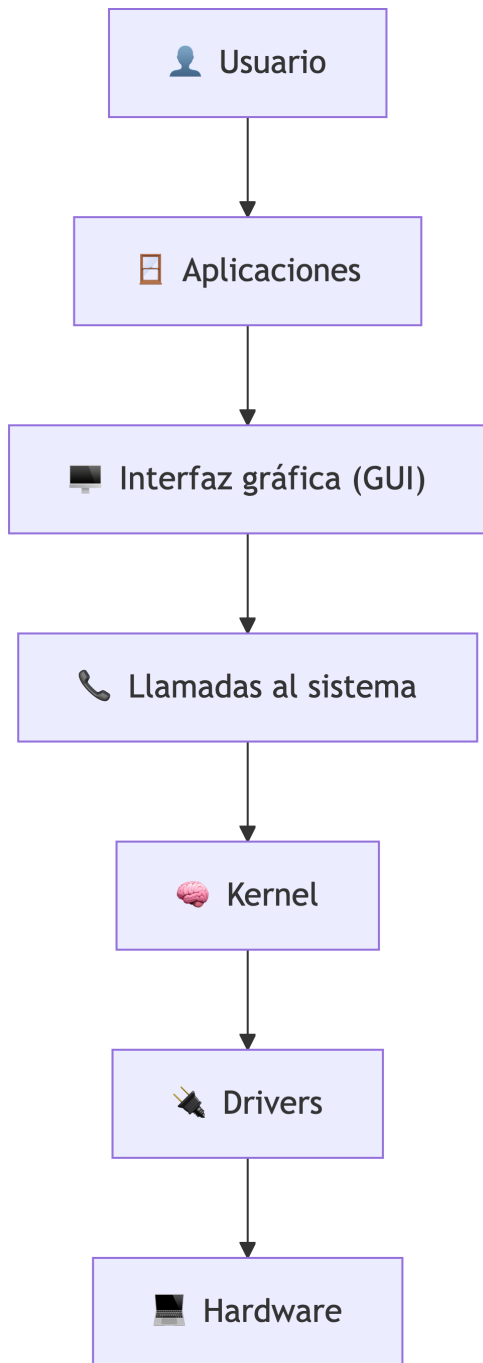
Ahora estudiaremos cada una de las capas comenzando desde la más cercana al usuario hasta llegar al **Kernel**, el componente más importante del Sistema Operativo.

Conociendo las capas del Sistema Operativo

En la sección anterior vimos que una solicitud recorre varias capas antes de llegar al hardware.

Ahora estudiaremos cada una de ellas para comprender su función.

Vista general



Cada capa tiene una responsabilidad específica y colabora con las demás.

Capa 1. Usuario

Es la persona que utiliza el computador.

Sus acciones son sencillas:

- Hacer clic.
- Escribir con el teclado.
- Abrir un programa.
- Guardar un archivo.

El usuario nunca controla directamente el hardware.

Capa 2. Aplicaciones

Son los programas que utilizamos diariamente.

Ejemplos:

- Google Chrome
- Microsoft Word
- Visual Studio Code
- VLC

Las aplicaciones solicitan servicios al Sistema Operativo, pero no acceden directamente al disco, la memoria o el procesador.

Idea importante

Las aplicaciones dependen del Sistema Operativo para utilizar los recursos del computador.

Capa 3. Interfaz gráfica (GUI)

La GUI permite que el usuario interactúe mediante:

- Ventanas.
- Botones.
- Menús.
- Iconos.

Gracias a ella no es necesario escribir comandos para realizar la mayoría de las tareas.

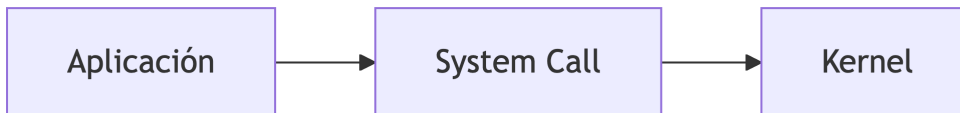
Capa 4. Llamadas al Sistema (System Calls)

Cuando una aplicación necesita un recurso, realiza una **llamada al sistema**.

Por ejemplo:

- Abrir un archivo.
- Reservar memoria.
- Crear un proceso.
- Imprimir un documento.

Estas solicitudes son enviadas al Kernel.



💡 Reflexiona

Cuando haces clic en **Guardar**, la aplicación no escribe directamente en el disco. Primero solicita ese servicio al Sistema Operativo mediante una llamada al sistema.

¿Qué observamos hasta ahora?

Las primeras capas están orientadas al usuario y a las aplicaciones.

Sin embargo, todavía no hemos llegado al componente que realmente controla el Sistema Operativo.

En la siguiente sección estudiaremos el **Kernel**, considerado el corazón del Sistema Operativo.

El Kernel: el corazón del Sistema Operativo

Hasta este momento hemos recorrido varias capas del Sistema Operativo.

Sin embargo, todas ellas dependen de un componente central.

Ese componente recibe el nombre de **Kernel**.

Si el Sistema Operativo fuera un cuerpo humano, el Kernel sería su **corazón y su cerebro**, ya que coordina el funcionamiento de todos los recursos del computador.

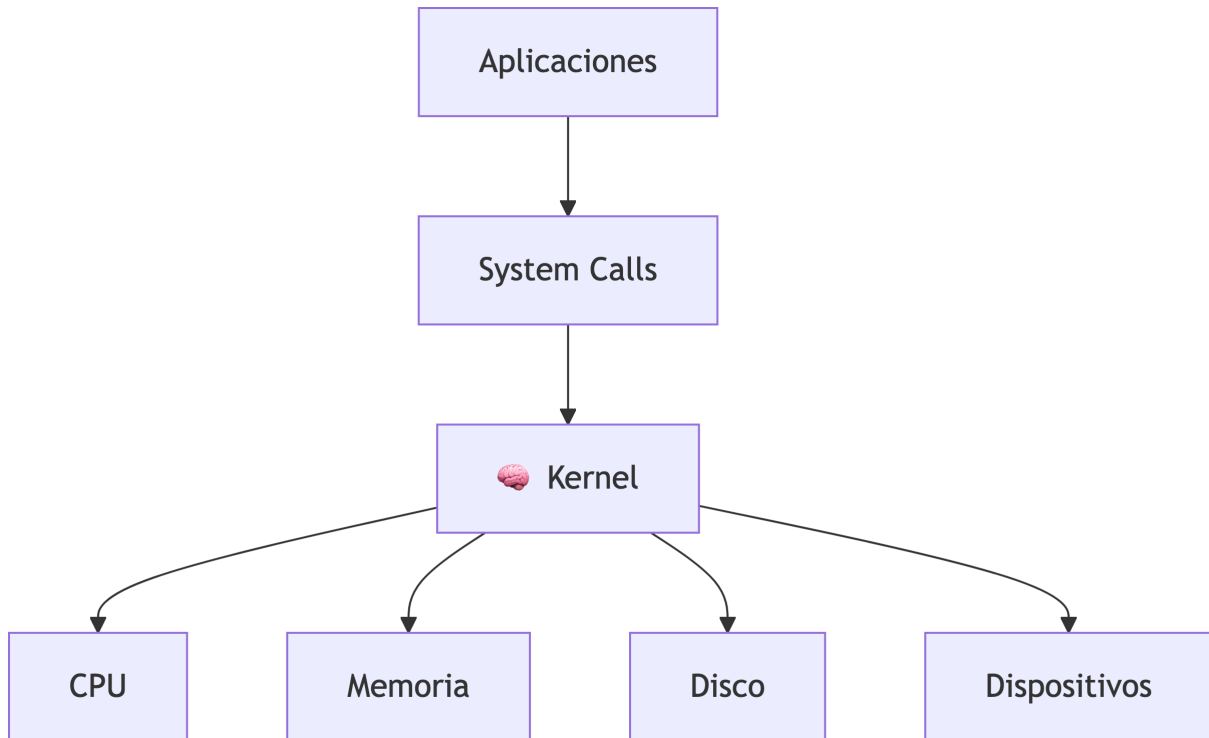
¿Qué es el Kernel?

El **Kernel** es la parte más importante del Sistema Operativo.

Es el primer componente que se carga en memoria durante el arranque del sistema y permanece ejecutándose mientras la computadora está encendida.

Su función principal es actuar como intermediario entre:

- Las aplicaciones.
- El hardware.
- Los dispositivos de entrada y salida.
- La memoria.
- El procesador.



i Idea clave

Las aplicaciones **nunca acceden directamente al hardware**. Todas las solicitudes pasan primero por el Kernel.

¿Qué responsabilidades tiene el Kernel?

El Kernel realiza numerosas tareas al mismo tiempo.

Entre las más importantes se encuentran:

- Administrar los procesos.
- Planificar el uso del procesador.
- Administrar la memoria.
- Controlar los dispositivos mediante los drivers.
- Gestionar el sistema de archivos.
- Atender interrupciones.

- Proteger los recursos del sistema.

Observa que muchos de estos temas serán estudiados durante el resto del curso.

¿Qué ocurriría si el Kernel no existiera?

Imagina que una aplicación intenta:

- Leer un archivo.
- Utilizar la impresora.
- Acceder a la memoria.
- Escribir información en el disco.

Sin el Kernel:

- Dos programas podrían utilizar la misma memoria.
- Varios procesos intentarían controlar el CPU al mismo tiempo.
- Los dispositivos no podrían coordinarse.
- El sistema sería inestable y probablemente dejaría de funcionar.

Reflexiona

El Kernel no solamente administra recursos.
También evita que los programas interfieran entre sí.

Relación con los temas estudiados

Observa cómo el Kernel aparece en casi todo lo que hemos aprendido.

Tema	Participación del Kernel
Procesos	Los crea y administra
Scheduler	Coordina la planificación
Dispatcher	Coordina el cambio de contexto
Memoria	Reserva y libera espacios
Archivos	Controla el acceso al disco

Tema	Participación del Kernel
Dispositivos	Se comunica mediante drivers

Esto demuestra por qué se le considera el **corazón del Sistema Operativo**.

Preparándonos para la siguiente sección

Ya conocemos la importancia del Kernel.

Ahora estudiaremos que **no todos los Sistemas Operativos utilizan el mismo tipo de Kernel**.

Analizaremos los modelos:

- Monolítico.
- Microkernel.
- Híbrido.

Y conoceremos cuál utiliza Windows, Linux y macOS.

Tipos de Kernel

En la sección anterior aprendimos que el **Kernel** es el corazón del Sistema Operativo.

Sin embargo, no todos los Sistemas Operativos implementan el Kernel de la misma forma.

Dependiendo de su diseño, existen diferentes arquitecturas.

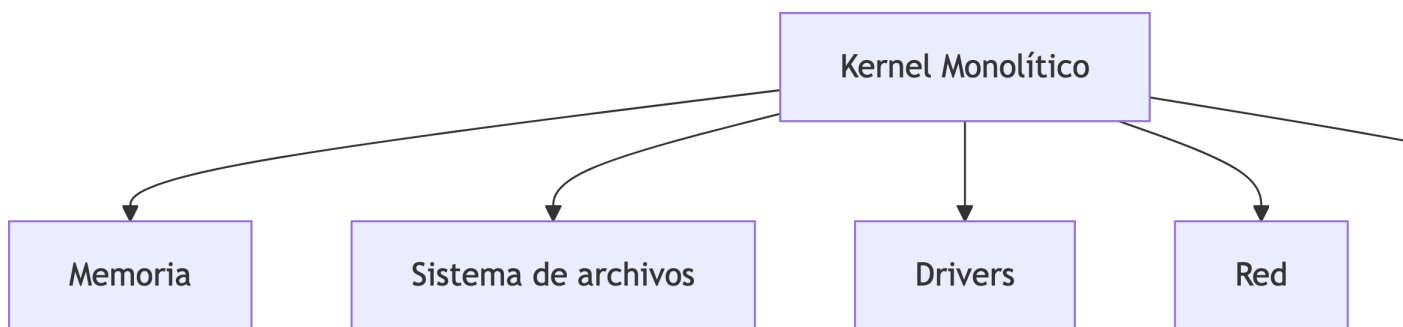
Kernel Monolítico

En un **Kernel Monolítico**, la mayoría de los servicios del Sistema Operativo se ejecutan dentro del propio Kernel.

Esto significa que:

- Administración de memoria.
- Sistema de archivos.
- Controladores (drivers).
- Redes.
- Planificación de procesos.

trabajan en un mismo espacio del Kernel.



Ventajas

- Muy rápido.
- Excelente rendimiento.
- Menor sobrecarga.

Desventajas

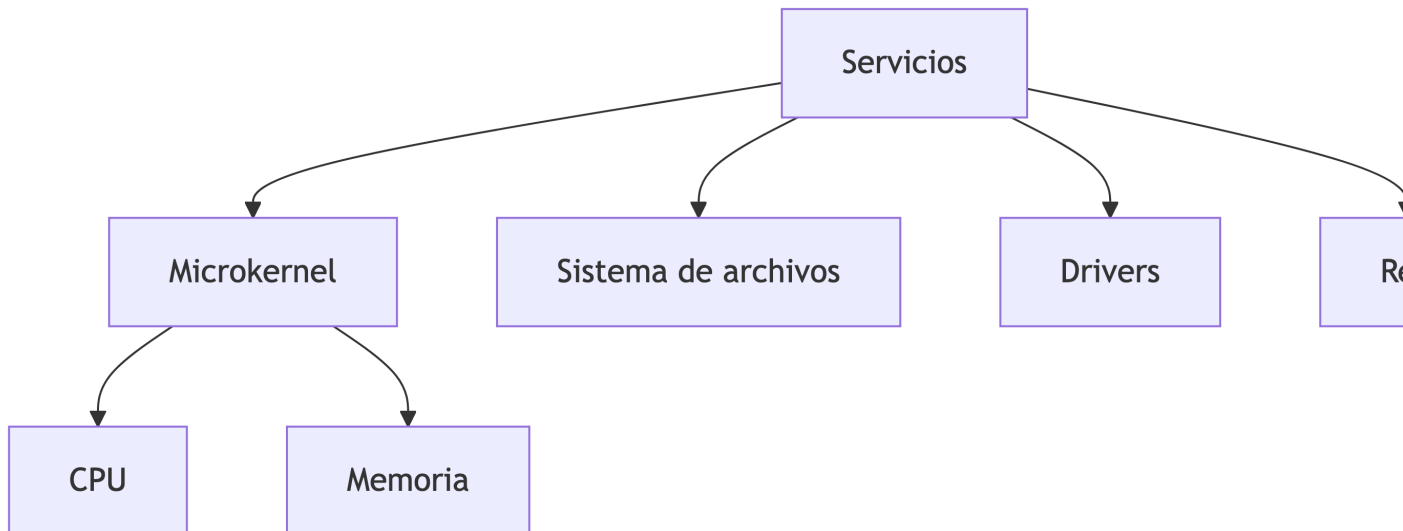
- Un error en un componente puede afectar todo el sistema.
- Mayor complejidad para mantener.

Ejemplo: **Linux**.

Microkernel

El **Microkernel** mantiene dentro del Kernel únicamente las funciones esenciales.

Otros servicios se ejecutan fuera del Kernel como procesos independientes.



Ventajas

- Mayor estabilidad.
- Más seguro.
- Fácil de mantener.

Desventajas

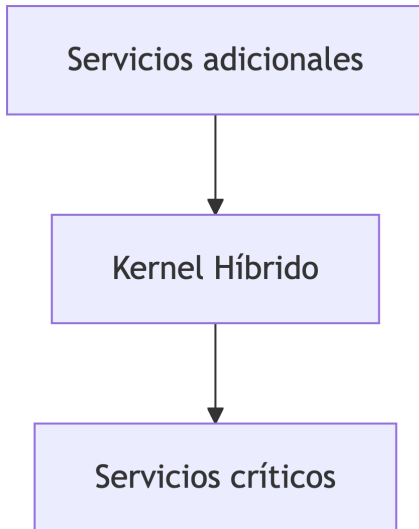
- Mayor comunicación entre procesos.
- Menor rendimiento que un monolítico.

Ejemplo clásico: **MINIX**.

Kernel Híbrido

El **Kernel Híbrido** combina características del Kernel Monolítico y del Microkernel.

Mantiene servicios importantes dentro del Kernel para mejorar el rendimiento y deja otros componentes fuera para aumentar la modularidad.



Ventajas

- Buen equilibrio entre rendimiento y estabilidad.
- Muy utilizado en sistemas modernos.

Desventajas

- Arquitectura más compleja.

Ejemplos:

- Windows
- macOS

Comparación

Característica	Monolítico	Microkernel	Híbrido
Rendimiento	Alto	Medio	Alto
Estabilidad	Media	Alta	Alta
Mantenimiento	Difícil	Fácil	Medio
Ejemplo	Linux	MINIX	Windows / macOS

i Idea clave

No existe un tipo de Kernel perfecto.

Cada arquitectura representa un equilibrio distinto entre **rendimiento**, **seguridad**, **mantenimiento** y **modularidad**.

Preparándonos para el cierre

En la siguiente sección seguiremos el recorrido completo de una aplicación desde que el usuario hace doble clic hasta que aparece en pantalla y resumiremos todo lo aprendido en este capítulo.

Caso integrador: ¿Qué ocurre cuando abrimos una aplicación?

Llegó el momento de integrar todo lo aprendido.

Imagina que haces doble clic sobre **Google Chrome**.

A simple vista únicamente aparece una ventana en la pantalla.

Sin embargo, internamente ocurre una secuencia de eventos perfectamente organizada.

Paso 1. El usuario solicita abrir la aplicación

El usuario hace doble clic sobre el icono.

La solicitud llega a la **Interfaz Gráfica (GUI)**.

Paso 2. La aplicación solicita servicios

La aplicación necesita:

- Leer su archivo ejecutable.
- Reservar memoria.
- Crear un proceso.
- Obtener tiempo de CPU.

Para ello realiza **llamadas al sistema (System Calls)**.

Paso 3. El Kernel entra en acción

El Kernel recibe la solicitud y comienza a coordinar todos los recursos necesarios.

Entre otras tareas:

- Crea el proceso.
- Reserva memoria.
- Consulta el sistema de archivos.
- Coordina el Scheduler.
- Se comunica con los Drivers.

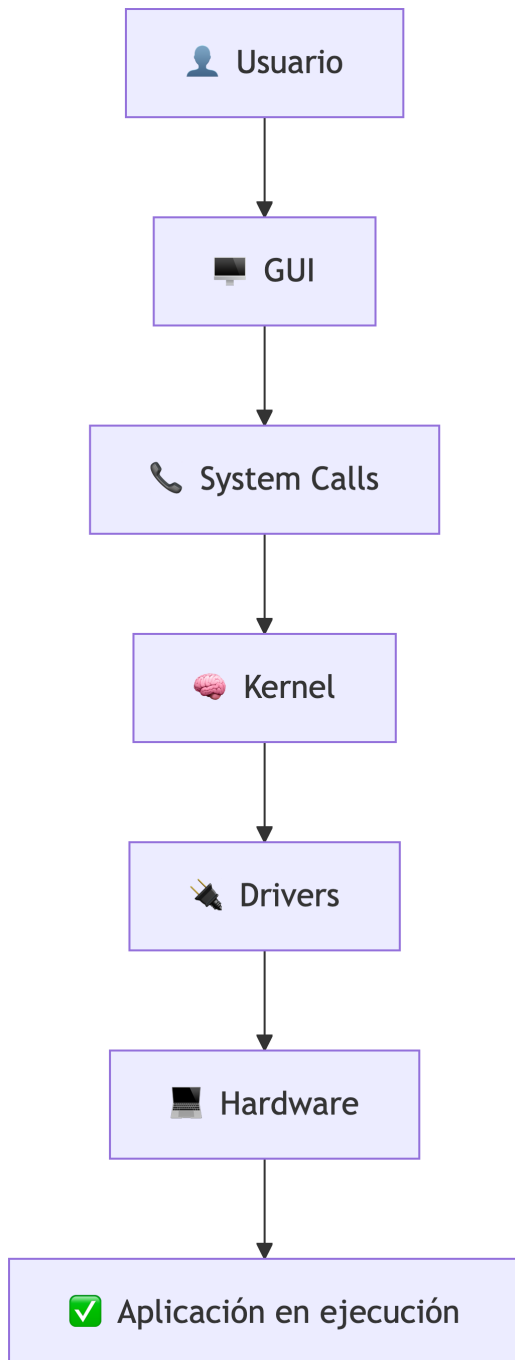
Paso 4. Los Drivers controlan el hardware

Los Drivers permiten que el Sistema Operativo se comunique con:

- Disco duro o SSD.
- Tarjeta gráfica.
- Teclado.
- Mouse.
- Monitor.

Paso 5. La aplicación aparece en pantalla

Cuando todos los recursos están preparados, la ventana finalmente aparece y el usuario puede comenzar a trabajar.



Idea clave

El usuario observa un único clic.

El Sistema Operativo coordina cientos de operaciones en apenas unos milisegundos.

Autoevaluación

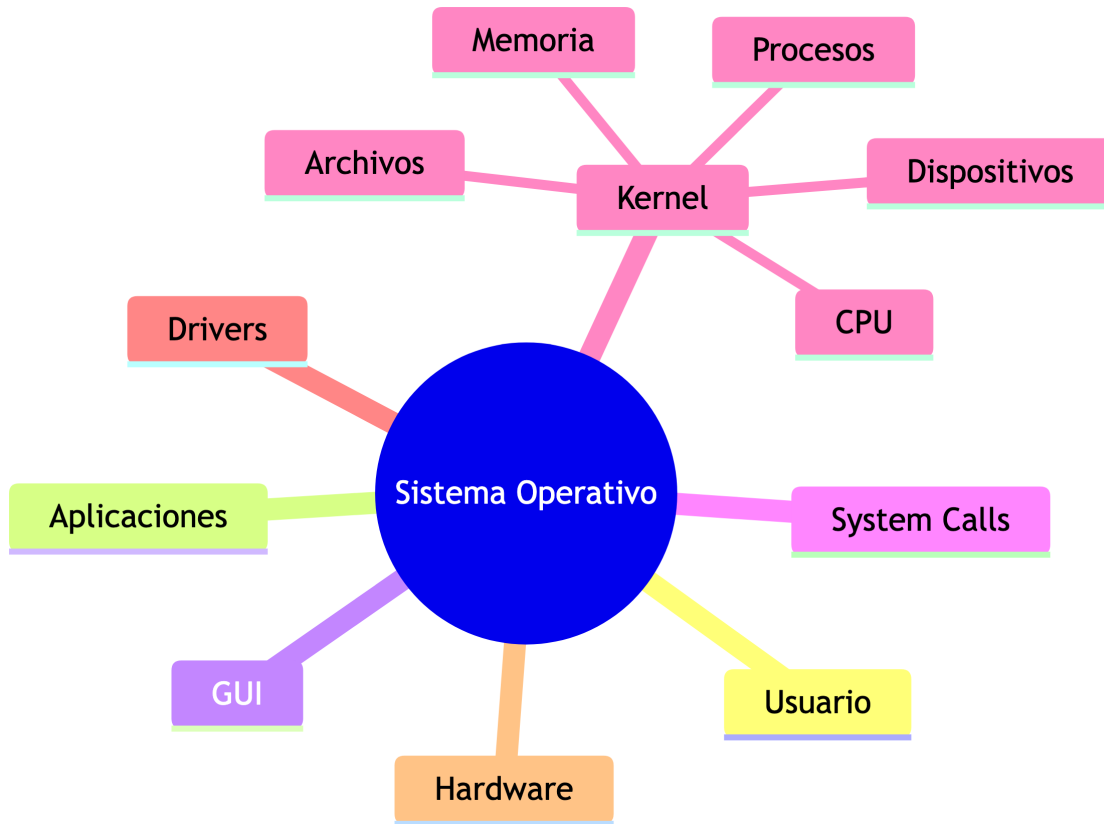
1. ¿Cuál es la función principal del Kernel?
 2. ¿Qué capa recibe directamente las solicitudes de las aplicaciones?
 3. ¿Qué son las System Calls?
 4. ¿Por qué las aplicaciones no acceden directamente al hardware?
 5. ¿Cuál es la función de los Drivers?
 6. ¿Qué diferencia existe entre un Kernel Monolítico y un Microkernel?
 7. ¿Qué ventajas ofrece un Kernel Híbrido?
 8. ¿Qué ocurriría si el Kernel dejara de funcionar?
 9. ¿Por qué decimos que el Kernel es el corazón del Sistema Operativo?
 10. Describe el recorrido que sigue una solicitud desde el usuario hasta el hardware.
-

Resumen del capítulo

Durante este capítulo comprendimos que un Sistema Operativo está organizado por capas para facilitar la administración de los recursos del computador.

Aprendimos que:

- El usuario interactúa con las aplicaciones.
- Las aplicaciones solicitan servicios mediante llamadas al sistema.
- El Kernel coordina todos los recursos.
- Los Drivers permiten controlar el hardware.
- Existen diferentes arquitecturas de Kernel según las necesidades del sistema.



i Para recordar

El **Kernel** es el componente más importante del Sistema Operativo. Todo acceso al procesador, memoria, dispositivos y archivos pasa por él.

Conexión con el siguiente capítulo

Ya comprendemos cómo el Sistema Operativo organiza sus componentes y administra el procesador.

En el siguiente capítulo estudiaremos cómo administra otro recurso fundamental:

La memoria principal (RAM).

Descubriremos cómo el Sistema Operativo asigna, protege y libera memoria para que múltiples procesos puedan ejecutarse de forma segura y eficiente.